

UNIVERSIDAD TECNOLÓGICA NACIONAL

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

TRABAJO PRÁCTICO INTEGRADOR

SISTEMA DE GESTIÓN DE PEDIDOS: "FOOD STORE"

Materia: Programación III

Estudiante: Lahoz Piantanida, Cristian Daniel

Fecha de Entrega: 01/07/2026

TABLA DE CONTENIDOS

1. INTRODUCCIÓN	Pág. 3
2. MARCO TEÓRICO	Pág. 3
○ 2.1. Tecnologías del Backend (Ecosistema Spring)	Pág. 3
○ 2.2. Tecnologías del Frontend (Cliente Reactivo)	Pág. 4
○ 2.3. Base de Datos y Persistencia	Pág. 4
3. ARQUITECTURA Y DECISIONES TÉCNICAS	Pág. 5
○ 3.1. Patrón DTO (Data Transfer Object) y Desacoplamiento	Pág. 5
○ 3.2. Estrategia de Criptografía Centralizada	Pág. 5
○ 3.3. Manejo Global de Excepciones Perimetrales	Pág. 6
○ 3.4. Optimización de Consultas Relacionales	Pág. 6
4. DIFICULTADES ENCONTRADAS Y SOLUCIONES APLICADAS	Pág. 7
○ 4.1. El Conflicto de Unicidad en la Siembra de Datos	Pág. 7
○ 4.2. Descalces de Contratos en la Deserialización de JSON	Pág. 7
○ 4.3. Prevención de Riesgos Críticos de Seguridad	Pág. 8
5. INTERFACES DEL SISTEMA (MANUAL VISUAL)	Pág. 9
6. CONCLUSIÓN	Pág. 13
7. BIBLIOGRAFÍA Y WEBGRAFÍA	Pág. 14

1. Introducción

El presente proyecto, denominado "**Food Store**", consiste en el desarrollo e implementación de un ecosistema de software completo e integrado de punta a punta para un negocio de comercio electrónico gastronómico. La solución unifica una aplicación cliente reactiva y moderna con una API REST robusta que automatiza la persistencia relacional en disco. El objetivo pedagógico de este trabajo es aplicar los conceptos avanzados de inversión de control, programación orientada a capas, control transaccional, seguridad informática y manipulación dinámica de interfaces web sin depender de frameworks pesados en el cliente.

2. Marco Teórico

2.1. Tecnologías del Backend

- **Java 21 (LTS):** Se adoptó la última versión de soporte a largo plazo de Java para aprovechar características modernas de sintaxis, rendimiento optimizado en la Máquina Virtual de Java (JVM) y el uso nativo de tipos estructurados inmutables como los *Records* para el traslado de información.
- **Spring Boot 3.x:** Framework core utilizado bajo el paradigma de **Inversión de Control (IoC)** y **Inyección de Dependencias (DI)**. Spring Boot elimina la necesidad de configuraciones manuales complejas de infraestructura, proveyendo un servidor embebido Apache Tomcat y gestionando de forma transparente el ciclo de vida de los componentes mediante anotaciones como `@Service`, `@RestController` y `@Component`.
- **Spring Data JPA / Hibernate:** Implementación de la especificación de Persistencia de Java (JPA) encargada de mapear objetos del dominio orientados a objetos (*Entities*) hacia tablas relacionales de bases de datos. Automatiza operaciones CRUD genéricas, transacciones de base de datos (`@Transactional`) y ciclos de vida de entidades en memoria.

2.2. Tecnologías del Frontend

- **Vite:** Herramienta de empaquetado rápida de última generación que sustituye los antiguos flujos de compilación pesados. Provee un servidor de desarrollo ultrarrápido gracias al uso de módulos nativos de JavaScript (ESM) y Hot Module Replacement (HMR) para reflejar cambios en tiempo real.
- **TypeScript Vanilla:** Se prescindió de frameworks complejos como React o Angular en el cliente para demostrar un dominio profundo del lenguaje y de las APIs del navegador. TypeScript añade tipado estricto estático sobre JavaScript tradicional, permitiendo capturar errores de aserción en tiempo de compilación y definir contratos estrictos mediante **Interfaces**. La manipulación de la interfaz gráfica se ejecuta de manera directa mediante la manipulación del Modelo de Objetos del Documento (**DOM**).
- **Axios:** Cliente HTTP basado en promesas para el navegador. Se configuró mediante instancias centralizadas con interceptores capaces de inyectar de forma transparente cabeceras de autenticación (como **X-User-Id**) en cada solicitud perimetral al servidor gastronómico.

2.3. Base de Datos y Persistencia

- **H2 Database Engine:** Motor de base de datos relacional ligero escrito en Java. Para cumplir con las directivas de robustez académica, se configuró en modo embebido acoplado a persistencia física local (guardado en un archivo físico **.mv.db** dentro del directorio del proyecto), garantizando que los datos de productos, categorías, usuarios y comandas no se destruyan al reiniciar el servidor.

3. Arquitectura y Decisiones Técnicas

3.1. Patrón DTO (Data Transfer Object) y Desacoplamiento

Una de las decisiones arquitectónicas más crítica fue el aislamiento absoluto del modelo de datos de la base de datos respecto de los datos que viajan por la red web. Para ello se implementó el patrón **DTO** mediante el uso de *Java Records*.

Las entidades JPA (**User**, **Product**, **Order**) nunca se exponen de forma directa en los **@RestController**. El flujo se divide en:

- **Request DTOs:** Capturan y validan la información entrante del usuario (ej. **UserRequestDTO**).
- **Response DTOs:** Moldean exclusivamente la información de salida permitida para el cliente (ej. **UserResponseDTO**).

El mapeo bidireccional se delega en **MapStruct**, una librería de generación de código que inyecta mapeadores automáticos y eficientes en tiempo de compilación, configurada de manera defensiva con **ReportingPolicy.IGNORE** para omitir propiedades no deseadas.

3.2. Estrategia de Criptografía Centralizada

En la primera fase de desarrollo se planteó un esquema de hashing hexadecimal en el cliente. Sin embargo, para alinearse con los estándares modernos de la industria, se resolvió **centralizar la lógica criptográfica en el Backend**.

Se programó un componente especializado llamado **CustomPasswordEncoder** que encapsula algoritmos criptográficos robustos de un solo sentido (Hashing unidireccional). Cuando un usuario se registra o actualiza su perfil, el frontend despacha de manera segura la cadena en texto plano por la red (protegida bajo el perímetro HTTP), y el backend se encarga de interceptarla, codificarla y guardarla en la tabla. Al iniciar sesión, la clave provista se procesa bajo el mismo algoritmo y se compara contra el hash inmutable de la base de datos.

3.3. Manejo Global de Excepciones Perimetrales

Para evitar que fallos lógicos o excepciones nativas de la base de datos (como una violación de llave foránea o un elemento no encontrado) expongan trazas de la pila (*stack traces*) comprometiendo la seguridad del servidor y quebrando la experiencia del usuario, se implementó un interceptor global de excepciones mediante la anotación

`@RestControllerAdvice` en la clase `GlobalExceptionHandler`.

Este componente captura de forma centralizada excepciones personalizadas de negocio (`BusinessException`, `ResourceNotFoundException`) y errores de validación de Spring Validation (`MethodArgumentNotValidException`), transformándolos en un objeto plano estandarizado (`ErrorResponseDTO`) acompañado del código de estado HTTP semántico correcto (`404 NOT FOUND`, `400 BAD REQUEST`, etc.):

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponseDTO>
handleResourceNotFound(ResourceNotFoundException ex) {
    ErrorResponseDTO error = new ErrorResponseDTO("RECURSO_NO_ENCONTRADO",
ex.getMessage());
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
}
```

3.4. Optimización de Consultas Relacionales

Al listar los productos en el catálogo o en el panel administrativo, cada fila requiere mostrar el nombre semántico de su categoría asociada. En un esquema ORM tradicional, esto puede desencadenar el problema de rendimiento conocido como **Query N+1** (una consulta para listar los N productos y N consultas adicionales a la tabla de categorías por cada fila).

Para mitigar esto de raíz sin sobrecargar de complejidad la base de datos, se aplicó una estrategia de **Caché Eficiente en RAM** dentro de `ProductServiceImpl.java`. Al recuperar el catálogo, el servicio recolecta las categorías activas e inyecta un mapa indexado llave-valor (`Map<UUID, String>`) en memoria volátil. La resolución del nombre se ejecuta en tiempo constante **O(1)**, reduciendo drásticamente la latencia y la cantidad de viajes por red hacia la base de datos H2.

4. Dificultades Encontradas y Soluciones Aplicadas

4.1. El Conflicto de Unicidad en la Siembra de Datos

Dificultad: Al configurar la base de datos H2 con persistencia local persistente en archivo físico, la aplicación funcionaba correctamente en el primer arranque, pero en los reinicios sucesivos fallaba drásticamente lanzando excepciones de tipo `DataIntegrityViolationException` acompañadas del código de error nativo de SQL `23505` (*Violación de restricción de unicidad*). Esto ocurría porque los cargadores automáticos de datos semilla (`CommandLineRunner`) volvían a intentar inyectar las categorías por defecto y el usuario administrador base que ya existían físicamente en el disco.

Solución: Se reorganizó y encapsuló de forma estricta todo el flujo de siembra inicial de datos dentro del componente de infraestructura `UserLoad.java`. Se implementó una lógica defensiva **Idempotente**: antes de ejecutar cualquier sentencia de persistencia, el cargador ejecuta una verificación previa mediante el método `.count()`. Si los registros históricos existen en el archivo H2, el proceso salta la inserción limpiamente, garantizando un inicio en verde del servidor en cualquier escenario.

4.2. Descalces de Contratos en la Deserialización de JSON

Dificultad: Durante la integración de la pantalla de "Mis Pedidos" (historial del cliente), la interfaz del modal visual desplegaba de forma constante la leyenda *"No se encontraron productos registrados en esta transacción"*, a pesar de que el importe total era correcto. El error radicaba en un sutil descalce de nombres entre lenguajes: mientras que el backend de Spring Boot (Jackson) serializaba la colección en plural nativo (`orderDetails`), el frontend mapeaba interacciones asíncronas apuntando a la propiedad en singular (`dto.orderDetail`).

Solución: Se aplicó una reestructuración de interfaces en TypeScript separando estrictamente los contratos de envío de los contratos de recepción de datos. El `OrderRequestDTO` (POST) quedó configurado en singular conforme lo requiere el DTO receptor de Java, mientras que el `OrderResponseDTO` (GET) se sincronizó en plural con la clave de red auditada, resolviendo el flujo mediante la función de mapeo de dominio `mapToDomain`.

4.3. Prevención de Riesgos Críticos de Seguridad

Dificultad: Al implementar la Historia de Usuario requerida para la gestión completa de usuarios desde el panel administrativo, surgió un vector de riesgo crítico: un administrador logueado en la plataforma podía accidentalmente (o por fuerza bruta de una API externa) hacer clic en su propio botón de eliminación o eliminar al último administrador del sistema. Esto provocaba un bloqueo inmediato ("Apocalipsis de datos"), dejando al backend huérfano de accesos y obligando a intervenir las tablas SQL de forma manual.

Solución: Se implementó una barrera defensiva de doble validación. En el Frontend (`users.controller.ts`), se aplicó un filtro en memoria para excluir completamente a los usuarios de rol `ADMIN` de la grilla visual de borrado. Como protección perimetral definitiva en el Backend, se inyectó una regla transaccional estricta dentro del servicio `UserServiceImpl.java` que compara el ID a dar de baja contra el ID del administrador que ejecuta la acción, rechazando la operación de inmediato con una excepción controlada de negocio.

5. Interfaces del Sistema (Manual Visual de Capturas)

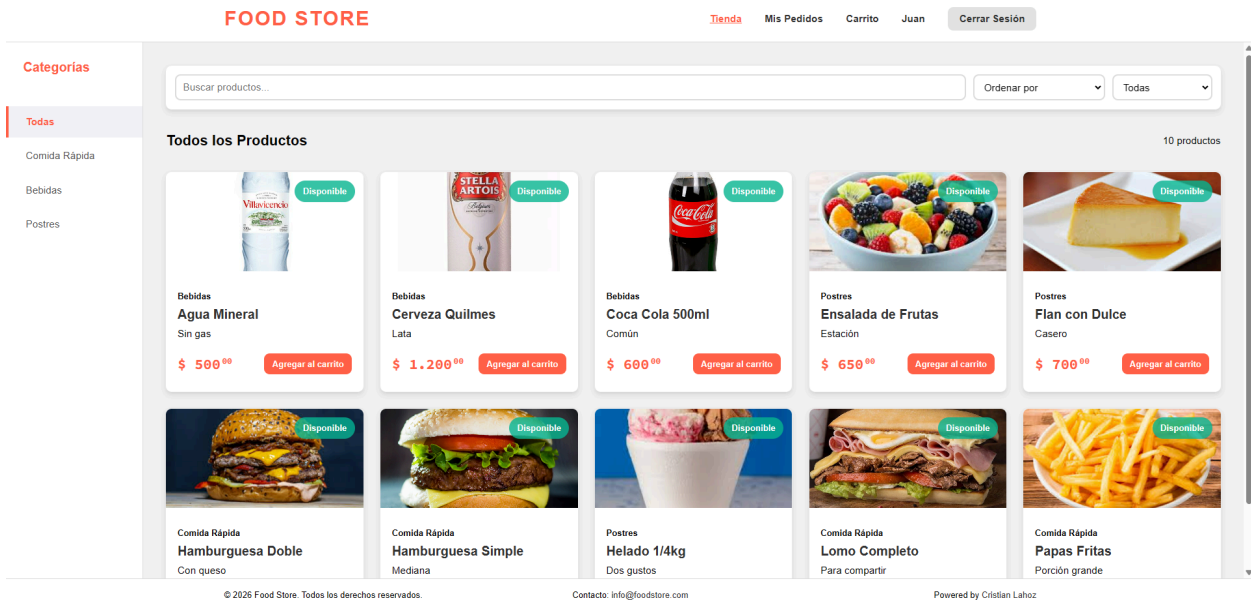


Figura 1: Catálogo Principal de la Tienda (Vista Cliente): Muestra la grilla reactiva de alimentos, las tarjetas de productos con sus respectivos precios formateados mediante la utilidad `formattedPriceHTML`, el sidebar izquierdo con los filtros por categorías dinámicas y la barra de filtros combinados.

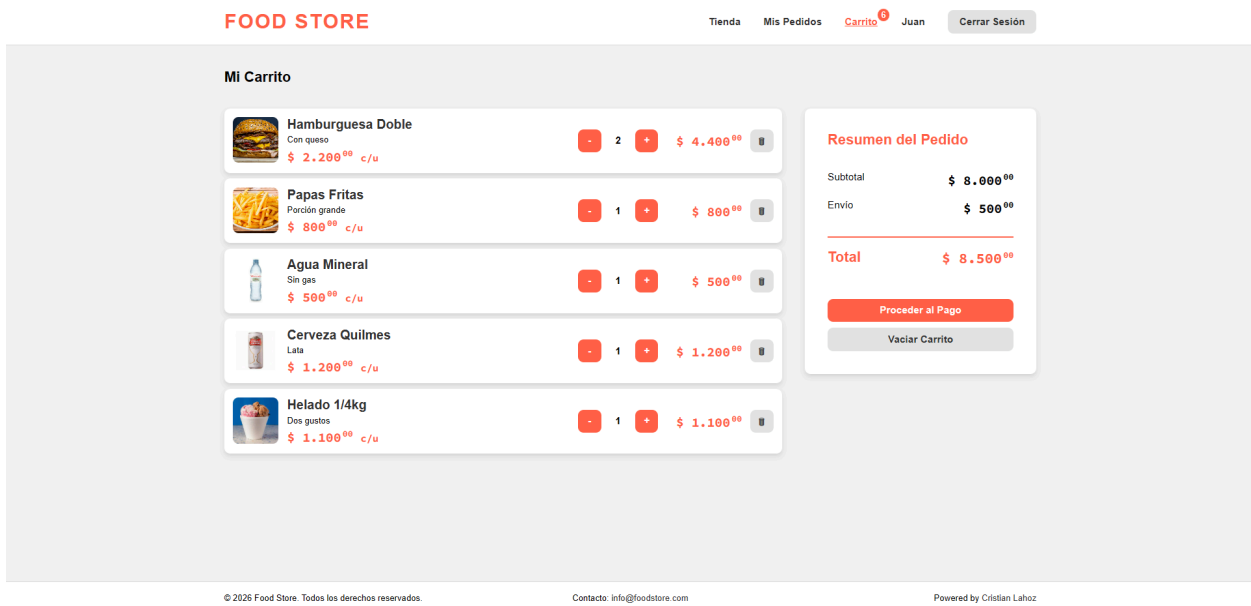


Figura 2: Carrito de Compras: Captura de los items del carrito con sus cantidades, subtotales y total calculados.

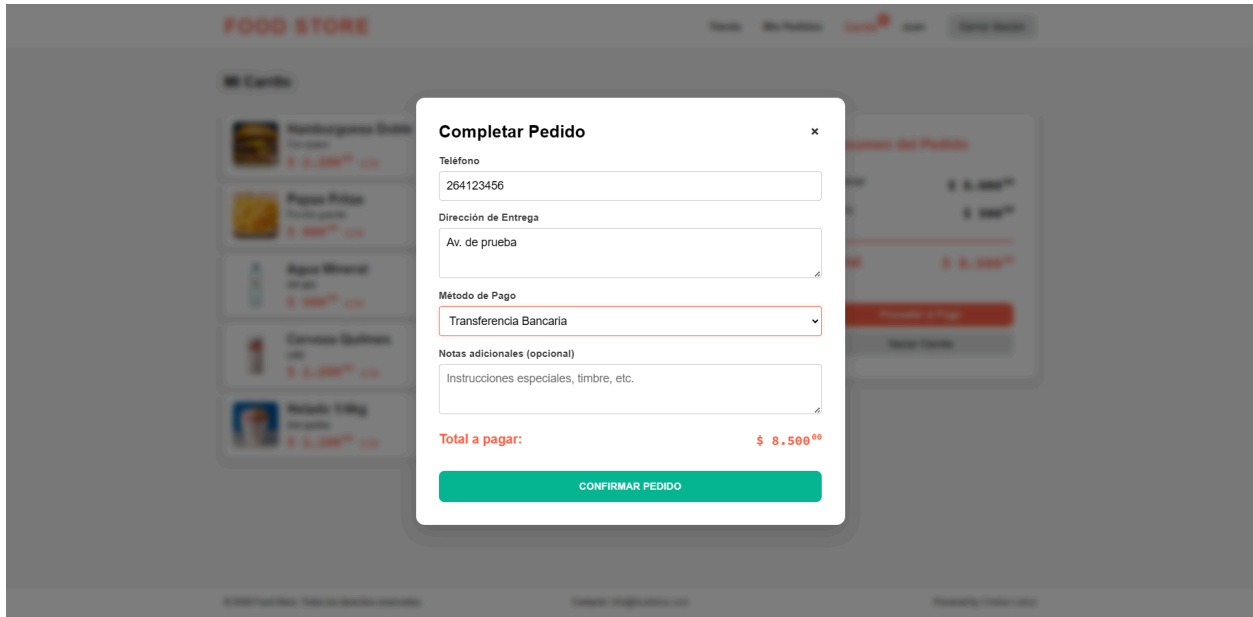


Figura 3: Checkout: Captura del panel flotante asíncrono que detalla las cantidades seleccionadas, el cálculo automático de subtotales y los inputs validados para registrar el teléfono, dirección de entrega y notas de la comanda gastronómica.

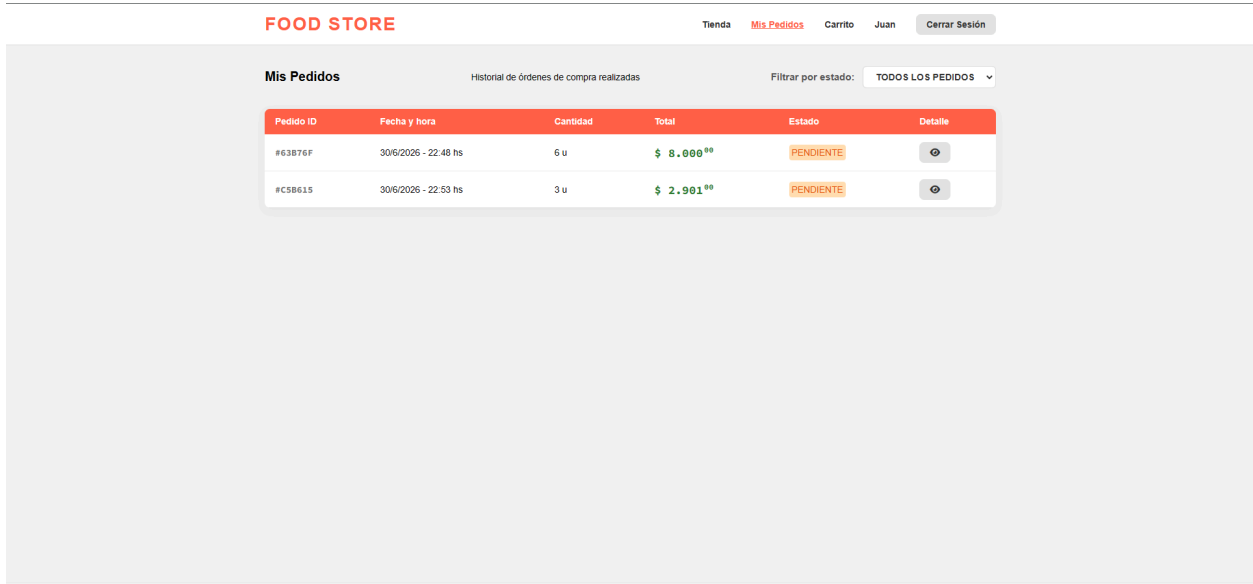


Figura 4: Historial de Compras ("Mis Pedidos"): Detalla la grilla donde el cliente sigue en tiempo real sus órdenes y visualiza las etiquetas de colores adaptadas por CSS según el estado logístico (PENDIENTE, CONFIRMADO, COMPLETADO, CANCELADO).

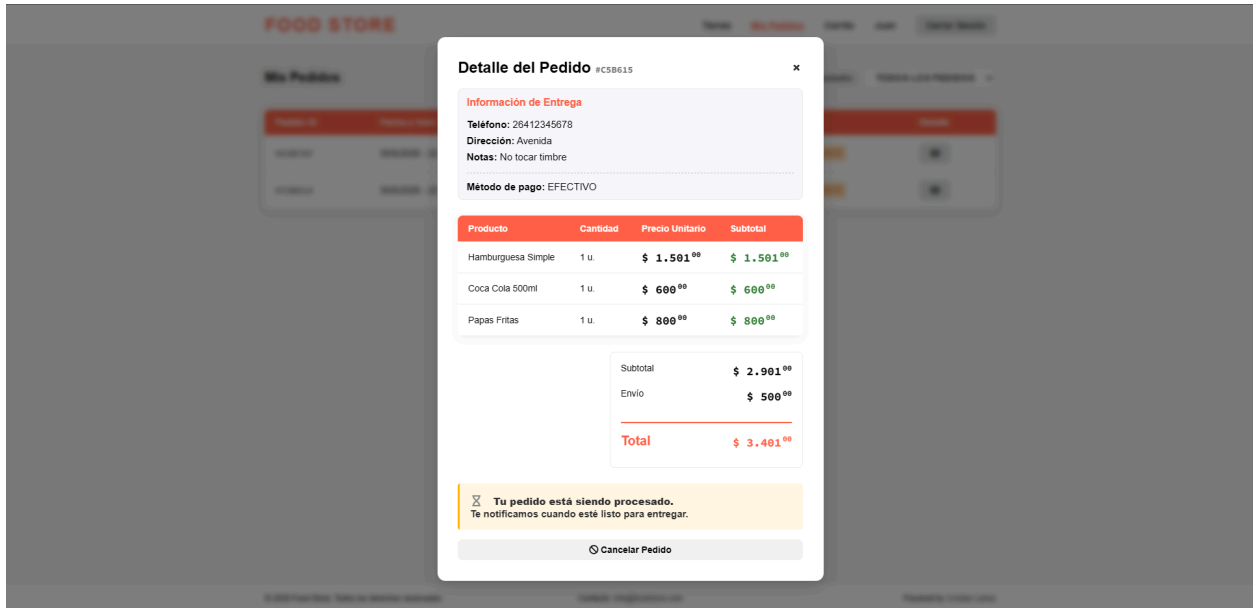


Figura 5: Modal de Desglose de Ítems (Con Botón de Cancelación Activo): Muestra el desglose interno de un pedido en estado pendiente, detallando cantidades y subtotales, e incorporando de forma condicional el botón de cancelación rápida de la Historia de Usuario.

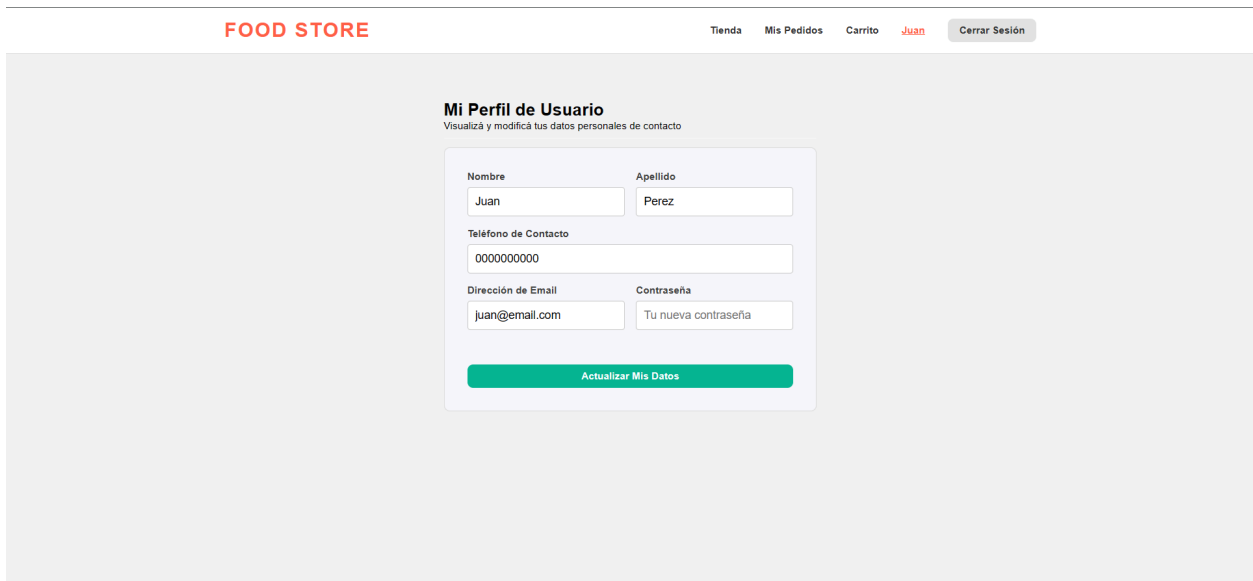


Figura 6: Formulario de Perfil de Usuario Actualizado: Captura de pantalla que refleja la interfaz de edición de datos de contacto personales, donde el campo de contraseña vacía opera de forma transparente protegiendo la clave actual en la base de datos relacional.

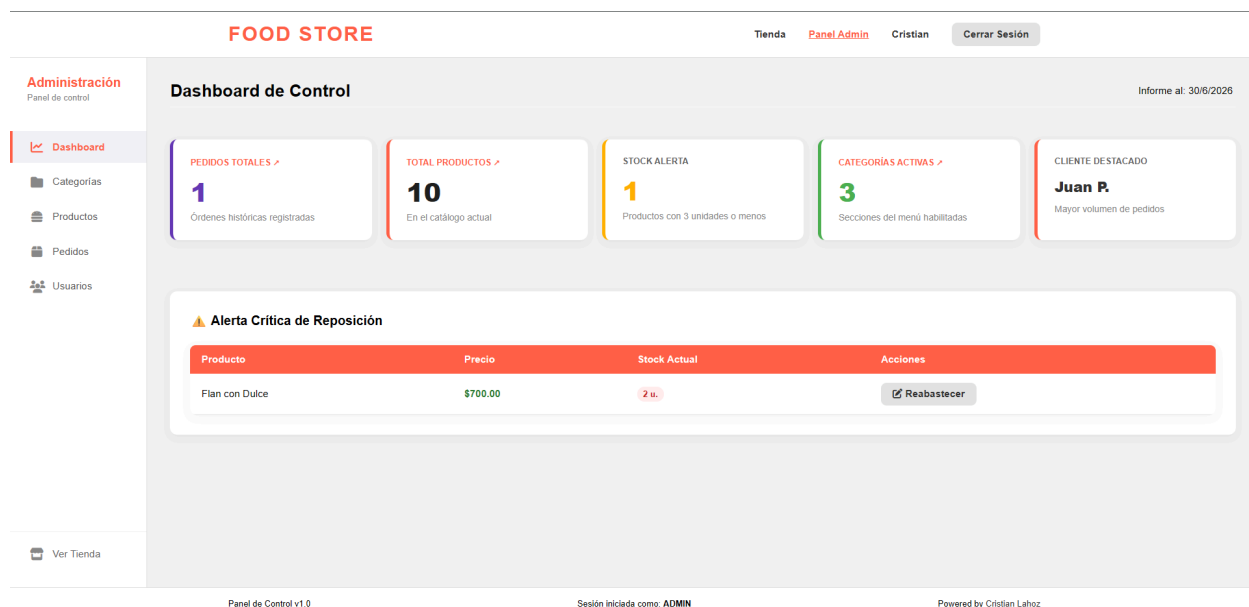


Figura 7: Dashboard (Vista Admin): Muestra las métricas, alerta de stock bajo, cliente destacado. Permite operar los CRUD de Productos, Categorías y actualizar estados de los pedidos.

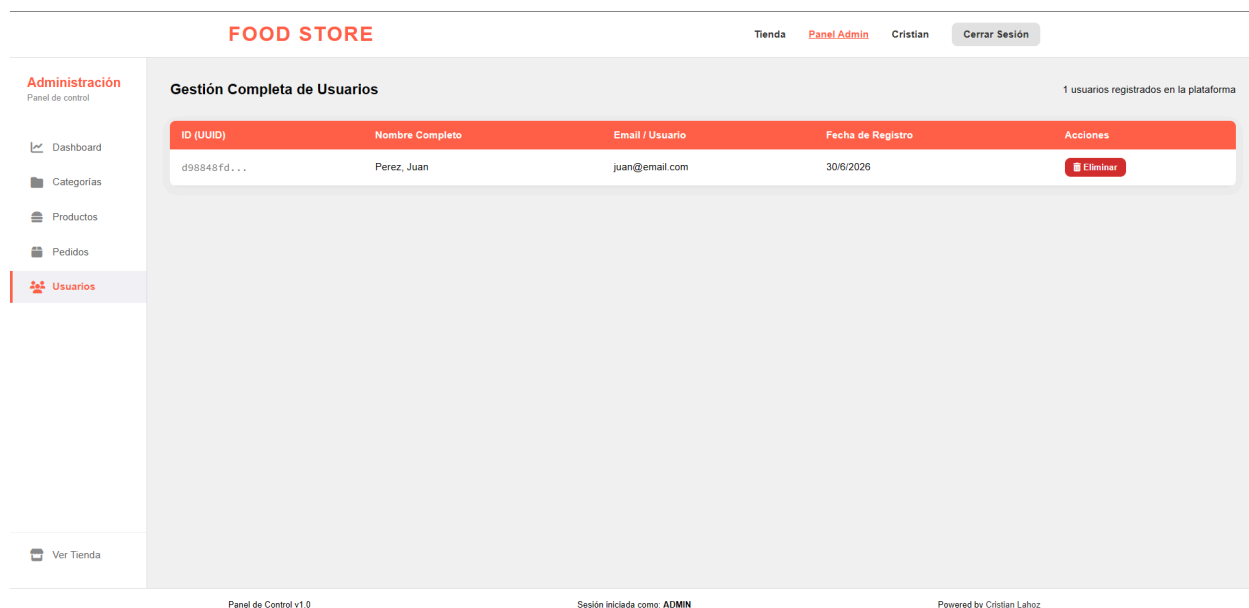


Figura 8: CRUD Administrativo de Usuarios (Vista Admin): Muestra el panel exclusivo del administrador donde se listan de forma filtrada únicamente las cuentas de clientes y se habilita el botón de baja lógica sin exponer hashes de contraseñas.

6. Conclusión

El desarrollo del ecosistema para **"Food Store"** ha permitido consolidar de forma práctica e integral los conceptos avanzados de ingeniería de software dictados en la materia

Programación III.

La separación absoluta de responsabilidades entre el Frontend y el Backend demostró las ventajas operativas del desacoplamiento tecnológico. Por un lado, la construcción de una Single Page Application (SPA) utilizando únicamente TypeScript Vanilla expuso la relevancia de una manipulación limpia y nativa del DOM y la correcta estructuración de servicios asíncronos mediante promesas, evitando la sobrecarga informática de frameworks pesados cuando la solución requiere una interfaz ágil y fluida.

Por otro lado, el Backend se consolidó como una fortaleza de reglas de negocio defensivas. La inmersión en el ecosistema de Spring Boot 3.x expuso el verdadero valor de la Inversión de Control (IoC), la gestión automatizada de contextos transaccionales y el poder de los mappers automatizados para resguardar la inmutabilidad de los datos web mediante Records DTOs. Haber enfrentado y resuelto desafíos del mundo real, tales como errores relacionales de bases de datos persistidas en disco, descalces informáticos de deserialización JSON y vectores de riesgo de seguridad en el borrado de cuentas, enriqueció profundamente el criterio técnico del estudiante. En conclusión, el proyecto no solo cumple con la totalidad de los requerimientos y la Definición de Completado de la cátedra, sino que sienta las bases de arquitectura limpia esenciales para el diseño futuro de sistemas comerciales escalables y profesionales.

7. Bibliografía y Webgrafía

1. **Spring Framework Documentation:** Guía de referencia oficial sobre Inyección de Dependencias, Contextos e Inversión de Control.
<https://docs.spring.io/spring-framework/reference/>
2. **Spring Data JPA Reference Guide:** Documentación oficial para el manejo de repositorios relacionales automatizados y transacciones.
<https://docs.spring.io/spring-data/jpa/reference/>
3. **TypeScript Manual - Omit & Partial Utility Types:** Guía oficial de Microsoft sobre operadores de transformación avanzada de tipos estáticos.
<https://www.typescriptlang.org/docs/handbook/utility-types.html>
4. **Vite Documentation:** Guía de optimización de empaquetado y entornos Single Page Application de última generación.
<https://vite.dev/guide/>
5. **MapStruct Reference Guide:** Documentación para la implementación de mappers eficientes de Beans en tiempo de compilación para Java.
<https://mapstruct.org/documentation/reference/html/>
6. **H2 Database Engine Docs:** Manual de referencia para la configuración de motores de bases de datos embebidos e indexación física en disco local.
<https://www.h2database.com/html/manual.html>